

Reviewer Pack — Minimal Rank of Free Entropy over Graphs vs Distinguishing T...

Entry 5ea7fdc16e4d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 06:58:55 UTC

Minimal Rank of Free Entropy over Graphs vs Distinguishing Tensor Width for BP_ReadTwice

Entry ID: 5ea7fdc16e4d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 06:58:55 UTC

1. Conjecture

Field A (mathematical branch): Free Probability Theory **Field B** (complexity object): Branching Program: read-once vs read-twice (BP)

Statement:

[‘For a given graph G , the free entropy of its edge set, $F(G)$, is related to the distinguishing tensor width for BP_readTwice as follows: $F(G) = O(\log(\rho(P)))$ where $\rho(P)$ is the distinguishing tensor width of a BP_readTwice P .’, ‘If there exists a BP_readTwice P with free entropy $F(G) \leq k$ and distinguishing tensor width $\rho(P) \geq n^c$ for some constant c , then such a P cannot exist for any graph G .’, ‘This conjecture holds when G is a random graph on n vertices and P is chosen uniformly at random among all BP_readTwice of size n .’]

Rationale (proposer’s reasoning):

[‘Free entropy provides a measure of the uncertainty or randomness in a system, while distinguishing tensor width quantifies the difficulty of distinguishing between different read-twice branching programs. The conjecture posits a connection between these two concepts, suggesting that free entropy can be used to characterize the complexity of BP_readTwice.’, ‘This bridge might expose structure by providing a new approach to analyzing the complexity of BP_readTwice in terms of graph theory and probability, potentially leading to new algorithms or lower bounds.’, ‘The relationship between free entropy and distinguishing tensor width could also provide insights into the behavior of random graphs and their applications in computational complexity.’]

Taxonomy category: BP_READTWICE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): eb1616d953638854

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for all $n \leq 40$ and at least 80% of randomly generated graphs G , the computed free entropy $F(G)$ satisfies $F(G) = O(\log(\rho(P)))$ with $\rho(P)$ being

the distinguishing tensor width, where P is a `BP_readTwice` chosen uniformly at random. The conjecture is falsified if any seed produces a counterexample where $F(G) > k$ or $\rho(P) < n^c$ for some constant c and $\text{support_fraction} < 0.8$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "free entropy" AND "BP_readTwice" AND graph" - "distinguishing tensor width" AND "branching program read-once vs read-twice" AND free probability" - "random graph" AND "minimum rank of free entropy" AND "tensor width"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_graph(n):
        graph = {i: set() for i in range(n)}
        for _ in range(n * (n - 1) // 2):
            u, v = random.sample(range(n), 2)
            if v not in graph[u]:
                graph[u].add(v)
                graph[v].add(u)
        return graph

    def free_entropy(graph):
        n = len(graph)
        edges = sum(len(neighbors) for neighbors in graph.values()) // 2
        entropy = -edges * math.log2(1 / (n * (n - 1) // 2))
        return entropy

    def distinguishing_tensor_width(n):
        # Simplified version for demonstration; actual implementation needed
        return n ** 0.5
```

```

n_values = [5, 10, 15, 20, 30, 40]
instances_tested = 0
conjecture_holds = True
counterexample = ""

for n in n_values:
    graph = generate_random_graph(n)
    F_G = free_entropy(graph)
    rho_P = distinguishing_tensor_width(n)

    if F_G > math.log(rho_P):
        instances_tested += 1
        conjecture_holds = False
        counterexample = f"Graph with n={n}, F(G)={F_G}, ρ(P)={rho_P}"

return {
    "metric_name": "Free Entropy vs Distinguishing Tensor Width",
    "metric_value": math.log(rho_P),
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value) ** 2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not res["conjecture_holds"] for res in results):
        first_failing_seed = next((seed for seed, res in zip(seeds, results) if not res["conjecture_holds"]))
        print(f"RESULT: FALSIFIED counterexample=\"{res['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

holds': False, 'counterexample': 'Graph with n=40, F(G)=4832.487147816054, ρ(P)=6.324555320336759'
TRIAL: {'metric_name': 'Free Entropy vs Distinguishing Tensor Width', 'metric_value': 1.8444397270569}
TRIAL: {'metric_name': 'Free Entropy vs Distinguishing Tensor Width', 'metric_value': 1.8444397270569}
TRIAL: {'metric_name': 'Free Entropy vs Distinguishing Tensor Width', 'metric_value': 1.8444397270569}
TRIAL: {'metric_name': 'Free Entropy vs Distinguishing Tensor Width', 'metric_value': 1.8444397270569}
TRIAL: {'metric_name': 'Free Entropy vs Distinguishing Tensor Width', 'metric_value': 1.8444397270569}

```

TRIAL: {'metric_name': 'Free Entropy vs Distinguishing Tensor Width', 'metric_value': 1.8444397270569
 TRIAL: {'metric_name': 'Free Entropy vs Distinguishing Tensor Width', 'metric_value': 1.8444397270569
 TRIAL: {'metric_name': 'Free Entropy vs Distinguishing Tensor Width', 'metric_value': 1.8444397270569

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test produced counterexamples where $F(G) > k$ or $p(P) < n^c$ for some constant c , violating the conjecture's conditions.
 | next: Investigate the counterexamples to understand the discrepancy between the conjecture and the experimental results.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12339
2	propose	ollama_remote	glm4:latest	0	0	13799
3	preregistration	ollama_remote	glm4:latest	0	0	6364
4	novelty	ollama_remote	glm4:latest	0	0	4847
5	novelty	ollama_remote	glm4:latest	0	0	5530
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12195
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11750
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12600
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8227
10	judge	ollama_remote	glm4:latest	0	0	8248

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 95900 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/5ea7fdc16e4d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5ea7fdc16e4d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/5ea7fdc16e4d.tar.gz` (if generated)